



Databases and its Applications

Relational Modelling

Pooja T S
Computer Applications

Databases and its Applications

Relational Modelling

SQL: DDL and DML Basics, Table Creation

Pooja T S

Computer Applications



Databases and its Applications

What is PostgreSQL?

- ▶ PostgreSQL is an advanced, open-source, object-relational database management system (ORDBMS).
- ▶ It extends the standard SQL language with modern capabilities.
- ▶ It supports both structured and semi-structured data (via JSONB).
- ▶ Developed at the University of California, Berkeley, as a successor to Ingres.
- ▶ Known for stability, extensibility, and full ACID compliance.



Databases and its Applications

Key Features of PostgreSQL

- ▶ PostgreSQL offers rich enterprise-grade features:
 - **ACID Compliance** – ensures reliable transactions.
 - **MVCC (Multi-Version Concurrency Control)** – allows concurrent reads and writes.
 - **Extensibility** – users can define new datatypes, functions, and operators.
 - **JSON / JSONB Support** – enables hybrid relational + document storage.
 - **Full-text Search, CTEs, Window Functions.**



Databases and its Applications

Why Choose PostgreSQL?

- ▶ PostgreSQL is chosen widely for:
 - Strict adherence to SQL standards (SQL:2023 compliant).
 - Performance on complex analytical workloads.
 - Strong data integrity and transaction control.
 - Excellent community and third-party support.
- ▶ Supported by popular ORM frameworks like SQLAlchemy, Hibernate, and Django ORM.



Databases and its Applications

PostgreSQL Client-Server Model

- ▶ PostgreSQL uses a **client-server model**.
- ▶ Each session consists of:
 - A **server process** called `postgres`, which:
 - Manages database files.
 - Accepts connections from client applications.
 - Performs database actions on behalf of clients.
 - The server can handle multiple concurrent clients by *starting a fork for each new client connection*.



Databases and its Applications

PostgreSQL Client Applications

- ▶ A **client (frontend)** application connects to the PostgreSQL server to send queries and receive results.
- ▶ Common client types include:
 - A text-oriented tool like `psql`.
 - A graphical interface (GUI) such as pgAdmin.
 - A web server that accesses the database for dynamic content.
 - A specialised database maintenance or monitoring tool.



Databases and its Applications

PostgreSQL Communication

- ▶ The client and server may run on the same machine or on different hosts.
- ▶ Communication occurs over a **TCP/IP network connection**.
- ▶ Ensure that:
 - Database files are accessible to the server.
 - Proper authentication and access permissions are configured at both ends.
- ▶ This distributed design enables PostgreSQL to be scalable, secure, and platform-independent.



Databases and its Applications

Architecture Overview

- ▶ PostgreSQL follows a client-server architecture:
 - **Postmaster Process** – manages child processes and connections.
 - **Backend Processes** – handle SQL queries.
 - **Shared Buffers** – store recently accessed data blocks.
 - **Write-Ahead Log (WAL)** – ensures crash recovery and durability.
- ▶ Client programs (like `psql`, `pgAdmin`) communicate via TCP/IP.



Databases and its Applications

PostgreSQL Ecosystem

- ▶ Core tools and utilities include:
 - `psql` – interactive shell for running SQL commands.
 - `pgAdmin` – graphical client for database management.
 - `pg_dump`, `pg_restore` – backup and restore utilities.
 - `pg_cli` – enhanced command-line tool (optional).
- ▶ PostgreSQL also integrates with extensions such as:
 - PostGIS (geospatial)
 - pgVector (AI embeddings)



Databases and its Applications

Installing PostgreSQL

► Installation methods differ by OS:

- Windows – via EnterpriseDB Installer.
- macOS – via Homebrew or Postgres.app.
- Linux – using package manager:

```
sudo apt install postgresql postgresql-contrib
```

► After installation:

- Start service: `sudo service postgresql start`
- Enter shell: `sudo -i -u postgres`



Databases and its Applications

Accessing PostgreSQL via psql

▶ psql is the command-line client for PostgreSQL.

▶ Common meta-commands:

- \l - list all databases
- \c dbname - connect to database
- \dt - list tables
- \d table - describe table structure
- \q - quit

▶ Example: postgres=# \l

List of databases

Name	Owner	Encoding	Collate
------	-------	----------	---------

postgres	postgres	UTF8	en_US.UTF-8
----------	----------	------	-------------

template1	postgres	UTF8	en_US.UTF-8
-----------	----------	------	-------------



Databases and its Applications

Creating a Database and Connecting

- ▶ Syntax: `CREATE DATABASE database_name;`
- ▶ Example: `postgres=# CREATE DATABASE college_db;`
`CREATE DATABASE`
`postgres=# \c college_db`
You are now connected to database "college_db" as user "postgres".
- ▶ Database is now ready for table creation.



Databases and its Applications

PostgreSQL Datatypes – Overview

- ▶ Common PostgreSQL datatypes:
 - **Character Types:** CHAR(n), VARCHAR(n), TEXT
 - **Numeric Types:** SMALLINT, INTEGER, BIGINT, NUMERIC(p,s), SERIAL
 - **Date/Time Types:** DATE, TIME, TIMESTAMP, INTERVAL
 - **Boolean:** TRUE / FALSE
 - **Array, JSON, JSONB, UUID**
- ▶ PostgreSQL also supports custom user-defined types.



Databases and its Applications

System Catalogs in PostgreSQL

- ▶ PostgreSQL maintains internal catalogs for metadata:
 - `pg_class` - information on tables.
 - `pg_attribute` - columns and datatypes.
 - `pg_constraint` - constraints and rules.
- ▶ They are queryable like normal tables:

```
SELECT relname  
FROM pg_class WHERE relkind='r' AND relname NOT  
LIKE 'pg_
```



```
college_db=# CREATE TABLE department(deptid
SERIAL PRIMARY KEY, deptname VARCHAR(40));
CREATE TABLE
college_db=# INSERT INTO department(deptname)
VALUES ('Computer Science'), ('Mathematics');
INSERT 0 2
college_db=# SELECT * FROM department;
deptid | deptname
-----+-----
1 | Computer Science
2 | Mathematics
(2 rows)
```




Databases and its Applications

User Roles and Access Control

- ▶ PostgreSQL uses a role-based access model.
- ▶ Commands:
 - `CREATE ROLE role_name LOGIN PASSWORD 'pwd';`
 - `GRANT ALL PRIVILEGES ON DATABASE college_db TO role_name;`
- ▶ Roles can represent users or groups.
- ▶ Authentication handled via `pg_hba.conf`.



Databases and its Applications

Comparison with Other SQL Databases

- ▶ PostgreSQL vs. MySQL vs. SQL Server:
 - PostgreSQL – standards-compliant, extensible, supports advanced data types.
 - MySQL – faster for simple queries but limited in subqueries and transactions.
 - SQL Server – enterprise-grade, but proprietary and costlier.
- ▶ PostgreSQL balances performance, flexibility, and openness.



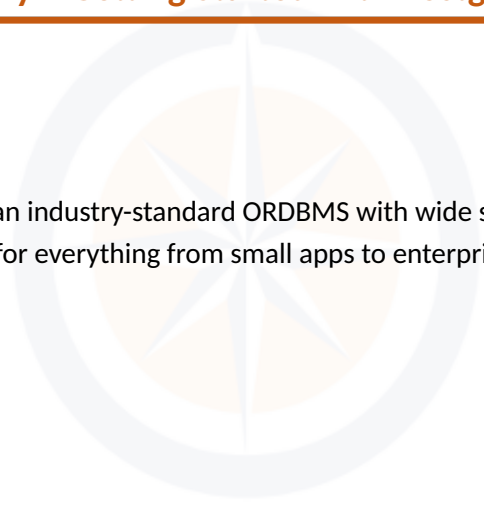
Databases and its Applications

Advantages of PostgreSQL

- ▶ PostgreSQL stands out because:
 - Open-source, free, and highly reliable.
 - Advanced query capabilities and indexing.
 - Excellent concurrency control (MVCC).
 - Active development community.
- ▶ Ideal for analytics, web backends, and data warehousing.



- ▶ PostgreSQL is an industry-standard ORDBMS with wide support.
- ▶ You can use it for everything from small apps to enterprise systems.





Databases and its Applications

Session Overview



- ▶ SQL allows defining, managing, and querying relational databases.
- ▶ This session focuses on two core sublanguages:
 - Data Definition Language (DDL)
 - Data Manipulation Language (DML)
- ▶ Practice database: `college_db`



Databases and its Applications

Understanding SQL Structure

- ▶ Every SQL command follows a consistent structure:
 - Syntax → `COMMAND target_object [clauses];`
 - Commands end with a semicolon (;).
 - SQL keywords are not case-sensitive.
 - PostgreSQL converts unquoted identifiers to lowercase by default.



Databases and its Applications

Case Sensitivity Rules in PostgreSQL

► Case rules:

- Unquoted identifiers → converted to lowercase.
- Quoted identifiers preserve case.

► Example:

```
CREATE TABLE Student(Name VARCHAR(30));
```

Column stored internally as `name`, not `Name`.

To preserve case, use quotes — `"Name"`.



Databases and its Applications

Data Definition Language (DDL)

- ▶ DDL defines and modifies the structure of a database.
- ▶ Works at the schema level, not on stored data.
- ▶ Core DDL commands include:
 - CREATE, ALTER, RENAME, DROP, TRUNCATE.
- ▶ In PostgreSQL, DDL statements are auto-committed (cannot be rolled back unless within a transaction).



Databases and its Applications

Why Do We Need DDL?

- ▶ DDL provides the foundation for all database structures:
 - Converts ER/EER diagrams into actual tables and relationships.
 - Creates tables, constraints, views, and indexes.
 - Ensures data integrity via constraints such as PRIMARY KEY and FOREIGN KEY.
 - Acts as the blueprint that DML operations act upon.



Databases and its Applications

CREATE DATABASE Command

- ▶ The CREATE DATABASE command creates a new database.

- ▶ Syntax:

```
CREATE DATABASE database_name;
```

- ▶ Example:

```
CREATE DATABASE college_db;
```

```
Connect using: \c college_db
```

- ▶ Output:

```
CREATE DATABASE
```

```
You are now connected to database "college_db" as  
user "postgres".
```



CREATE TABLE Command

- ▶ Used to create new tables and define columns, datatypes, and constraints.
- ▶ General Syntax:

```
CREATE TABLE table_name(  
    column1 datatype [constraint],  
    column2 datatype [constraint], ... );
```

- ▶ Common constraints:
 - PRIMARY KEY – Unique identifier.
 - UNIQUE – Prevents duplicate values.
 - NOT NULL – Field must contain a value.
 - CHECK – Restricts data based on a condition.
 - REFERENCES – Defines foreign key link.



CREATE TABLE Example – department

► Example:

```
CREATE TABLE department(  
    deptid SERIAL PRIMARY KEY,  
    deptname VARCHAR(50) UNIQUE NOT NULL);
```

► Output:

```
CREATE TABLE
```

► Verification:

```
\d department
```

Displays table columns, types, and constraints.



Databases and its Applications

CREATE TABLE Example – student

► Example:

```
CREATE TABLE student(  
  sid SERIAL PRIMARY KEY,  
  sname VARCHAR(40) NOT NULL,  
  deptid INT REFERENCES department(deptid),  
  cgpa NUMERIC(3,2) CHECK (cgpa BETWEEN 0 AND 10));
```

► Output:

```
CREATE TABLE
```

► Relation:

- student.deptid → department.deptid



Databases and its Applications

ALTER TABLE Command

- ▶ Used to modify the structure of an existing table without losing data.
- ▶ Syntax:

```
ALTER TABLE table_name action;
```
- ▶ Common actions:
 - ADD COLUMN – Adds a new attribute.
 - DROP COLUMN – Removes a column.
 - ALTER COLUMN – Changes datatype or default.
 - ADD CONSTRAINT – Adds PK/FK rules.



Databases and its Applications

ALTER TABLE Examples

- ▶ Add a new column:

```
ALTER TABLE student ADD COLUMN email VARCHAR(60);
```

- ▶ Modify datatype:

```
ALTER TABLE student ALTER COLUMN cgpa TYPE  
NUMERIC(4,2);
```

- ▶ Drop column:

```
ALTER TABLE student DROP COLUMN email;
```

- ▶ Output:

```
ALTER TABLE
```



Databases and its Applications

RENAME Command

- ▶ The RENAME operation is part of ALTER TABLE.

- ▶ Syntax:

```
ALTER TABLE old_name RENAME TO new_name;
```

- ▶ Example:

```
ALTER TABLE student RENAME TO learners;
```

- ▶ Output:

```
ALTER TABLE
```




Databases and its Applications

DROP and TRUNCATE Commands

- ▶ **DROP:** Permanently removes a table.

```
DROP TABLE learners;
```

- ▶ Output: DROP TABLE

- ▶ **TRUNCATE:** Deletes all rows instantly but retains structure.

```
TRUNCATE TABLE student;
```

- ▶ Output: TRUNCATE TABLE



Databases and its Applications

Data Manipulation Language (DML)

► What is DML?

- **Data Manipulation Language (DML)** consists of SQL commands that deal with the actual data stored inside tables.
- It allows **insertion, modification, retrieval, and deletion** of records.
- Common DML commands include:
INSERT, UPDATE, DELETE, and SELECT.
- Unlike DDL, DML does not change database structure — it operates on the data itself.



Databases and its Applications

Why Do We Need DML?

- ▶ DML enables practical interaction with stored data:
 - After defining the schema (via DDL), DML is used to **populate and maintain data**.
 - It acts as the main interface between end-users, applications, and the database engine.
 - It enables data-driven operations such as:
 - Adding new records using `INSERT`.
 - Making corrections or updates with `UPDATE`.
 - Removing obsolete or redundant entries using `DELETE`.
 - Viewing and analysing data using `SELECT`.
 - DML gives “life” to the database — without it, databases remain empty structures.



Databases and its Applications

When is DML Used?

- ▶ Typical scenarios where DML commands are applied:
 - **Initial data entry:** Populating tables after creation.
 - **Regular maintenance:** Updating or deleting data due to business rule changes.
 - **Querying:** Retrieving insights, reports, or analytics via SELECT.
 - **ETL operations:** Extracting and transforming data between systems.
 - **Transactional systems:** Performing atomic operations with COMMIT/ROLLBACK.



► Under the Hood: Multi-Version Concurrency Control (MVCC)

- PostgreSQL does not overwrite data directly; it creates new versions of rows.
- Each DML action affects the visibility of tuples:
 - INSERT – Adds a new tuple with a transaction ID (XID).
 - UPDATE – Creates a new version of a row and marks the old one as obsolete.
 - DELETE – Marks the tuple as deleted (invisible to new transactions).
- This design avoids locking conflicts and enables concurrent users to work safely.
- Vacuum and autovacuum processes later clean up obsolete versions.



Databases and its Applications

Types of DML Commands

- ▶ DML commands can be grouped as:
 - **INSERT** – Adds new data to tables.
 - **UPDATE** – Modifies existing records.
 - **DELETE** – Removes one or more records.
 - **SELECT** – Retrieves and filters data.
- ▶ All these operations are transaction-controlled — you can use **BEGIN**, **COMMIT**, or **ROLLBACK** for safety.



- ▶ **Purpose:** To insert one or more rows of data into a table.

- ▶ **Syntax:**

```
INSERT INTO table_name (col1, col2, ...)
VALUES (val1, val2, ...);
```

- ▶ **Example:**

```
INSERT INTO student (sname, deptid, cgpa)
VALUES ('Ravi', 1, 8.4);
```

- ▶ **PostgreSQL supports:**

- **Multi-row insertions**
- **Subquery insertions:** `INSERT INTO new_table SELECT *`
`FROM old_table;`



- ▶ **Purpose:** Modify one or more existing rows in a table.

- ▶ **Syntax:**

```
UPDATE table_name  
SET column1 = value1, column2 = value2  
WHERE condition;
```

- ▶ **Example:**

```
UPDATE student SET cgpa = 9.5 WHERE sname =  
'Ravi';
```

- ▶ **Without WHERE clause**, all records are modified — always use cautiously.



- ▶ **Purpose:** Remove one or more records from a table.

- ▶ **Syntax:**

```
DELETE FROM table_name WHERE condition;
```

- ▶ **Example:**

```
DELETE FROM student WHERE deptid = 2;
```

- ▶ **Note:**

- DELETE removes data row by row (logged operation).
- TRUNCATE clears all rows instantly (non-logged).



SELECT Command – The Core of DML

- ▶ **Purpose:** Retrieve data from one or more tables.

- ▶ **Syntax:**

```
SELECT column1, column2 FROM table_name WHERE  
condition;
```

- ▶ **Example:**

```
SELECT sname, cgpa FROM student WHERE cgpa > 8;
```

- ▶ PostgreSQL also supports:

- **Sorting:** ORDER BY
- **Filtering:** WHERE
- **Aggregation:** GROUP BY, HAVING



- ▶ DML commands form part of a transaction block:

```
BEGIN;
```

```
UPDATE student SET cgpa = 9.5 WHERE sname =  
'Ravi';
```

```
DELETE FROM student WHERE cgpa < 5.0;
```

```
COMMIT;
```

- ▶ If an error occurs before commit:

```
ROLLBACK; - undoes all DML changes.
```

- ▶ PostgreSQL guarantees **ACID compliance** — all DML operations are atomic and consistent.



Databases and its Applications

Performance and Locking in DML

- ▶ PostgreSQL DML uses row-level locks to ensure consistency:
 - Only the row being modified is locked, not the entire table.
 - Other transactions can still read previous versions (thanks to MVCC).
 - `SELECT FOR UPDATE` can be used for explicit row locking.
- ▶ DML performance tips:
 - Use indexes on columns appearing in `WHERE`.
 - Batch inserts for large data loads.
 - Avoid frequent `COMMITs` in loops; group transactions logically.



Databases and its Applications

Best Practices for Using DML

► Design and Execution Guidelines:

- Always validate with `SELECT` before executing `UPDATE` or `DELETE`.
- Use explicit `WHERE` conditions to avoid full-table changes.
- Prefer transactions (`BEGIN-COMMIT`) for grouped changes.
- Apply constraints and foreign keys to prevent orphaned data.
- Regularly run `VACUUM` or `autovacuum` for performance.
- Log critical DML operations in audit tables if required.



Thank You

Pooja T S

Assistant Professor

Department of Computer Applications

poojats@pes.edu

080-26721983 Extn: 233